

UNIVERSIDAD VERACRUZANA

Facultad de Estadística e Informática
Licenciatura en Ingeniería de Software

PRA08 – Pruebas End to End al MVP

PayFlow – Módulo de Pagos de Servicio Fijo

Experiencia Educativa:	Taller de Pruebas
Práctica:	PRA08 – Pruebas End to End
Equipo:	Gris
Integrantes:	Jorge Manuel Cobos Castro -- S23014045 Marcos Zenón Sánchez Mendizabal -- S23014093 Guillermo Velázquez Rosiles -- S23014116
Fecha de entrega:	24 de mayo de 2026

1. Introducción

Las pruebas **End-to-End (E2E)** en sistemas de educación financiera personal como **PayFlow** validan que las transacciones no solo sean matemáticamente correctas, sino que cumplan con las reglas de negocio desde que el usuario da clic en 'Pagar' hasta que el sistema emite el comprobante digital y actualiza el balance.

En esta práctica se simula el pago de un **Servicio Fijo** (Renta, Internet o Luz). El objetivo es diseñar, implementar y ejecutar una suite de pruebas E2E que valide el flujo completo del pago, integrando la validación de fondos, la aplicación de comisiones y la generación del folio de transacción, todo bajo el ciclo **TDD (Red-Green-Refactor)**.

1.1 Objetivos de la práctica

- **Diseñar:** Definir los escenarios E2E que cubran el flujo completo del pago, incluyendo camino feliz, rechazo por fondos insuficientes y concepto inválido.
- **Implementar:** Desarrollar `payflow_pago.py` con las tres capas (validación, cálculo y comprobante) y el orquestador `procesar_pago()`.
- **Ejecutar:** Correr la suite `test_pago_e2e.py` con `pytest` y alcanzar cobertura $\geq 90\%$ sobre el archivo de lógica.
- **Analizar:** Documentar el slicing de capas y responder cómo la comisión de \$15.00 se aplica en la Capa Inferior sin que la Capa Superior conozca la operación matemática.

2. Planteamiento del sistema

El MVP procesa el pago de un servicio fijo con la siguiente arquitectura de tres capas:

Capa	Función	Responsabilidad	No hace...
Superior (Validación)	<code>validar_pago()</code>	Verifica si el saldo cubre monto + \$15.00 de comisión. Comprueba que el concepto sea Renta, Internet o Luz.	No resta dinero. No genera folios.
Inferior (Balance)	<code>calcular_nuevo_saldo()</code>	Realiza la operación aritmética pura: saldo – monto – comisión. Recibe números, devuelve un número.	No valida reglas. No conoce el concepto.
Media (Comprobante)	<code>generar_folio()</code>	Construye la cadena PAGO- una vez confirmado el pago.	No decide si el pago procede.
Orquestador	<code>procesar_pago()</code>	Coordina las tres capas en una sola llamada: validar → calcular → formatear → devolver resultado.	Es la única función que el front-end llama.

2.1 Condiciones del sistema

Entrada: ID de usuario, concepto ("Renta", "Internet" o "Luz"), monto fijo del concepto, saldo disponible.

Comisión fija: \$15.00 por cada transacción de pago de servicio.

Fórmula de validación: $\text{saldo} \geq \text{monto} + 15.00$

Fórmula de balance: $\text{nuevo_saldo} = \text{saldo} - \text{monto} - 15.00$

Formato del folio: PAGO- [CONCEPTO] [TIMESTAMP] -- ej. PAGO-RENTA20260517143022

Inmutabilidad: Si el pago es rechazado por cualquier motivo, el saldo original no se modifica.

3. Matriz de Decisión -- Módulo de Pagos PRA08

La matriz enumera **todas las combinaciones posibles** de condiciones de entrada y el resultado esperado del sistema para cada una. Se aplica la técnica de **Tabla de Decisión** (ISO/IEC 29119-4) para garantizar que ninguna combinación lógica quede sin prueba.

3.1 Condiciones y acciones del sistema

ID	Condición	Descripción	Valores posibles
C1	Concepto válido	El concepto pertenece al catálogo: {Renta, Internet, Luz}	Sí / No
C2	Monto > 0	El monto a pagar es un número positivo mayor a cero	Sí / No
C3	Saldo suficiente	El saldo disponible cubre monto + \$15.00 de comisión	Sí / No

ID	Acción / Resultado	Descripción
A1	RECHAZADO – concepto inválido	El sistema rechaza antes de evaluar saldo. Saldo intacto. folio = None.
A2	RECHAZADO – monto inválido	El sistema rechaza por monto ≤ 0 . Saldo intacto. folio = None.
A3	RECHAZADO – fondos insuficientes	Validación pasa concepto y monto pero el saldo no alcanza. Saldo intacto. folio = None.
A4	APROBADO – pago procesado	Todas las condiciones se cumplen. nuevo_saldo = saldo – monto – 15. folio generado.

3.2 Tabla de Decisión completa

La siguiente tabla expande las 8 combinaciones lógicas posibles de las tres condiciones ($C1 \times C2 \times C3 = 2^3 = 8$ reglas). El sistema evalúa las condiciones en orden $C1 \rightarrow C2 \rightarrow C3$; si alguna falla, se cortocircuita y no se evalúan las siguientes.

Regla	C1 Concepto válido	C2 Monto > 0	C3 Saldo suficiente	Acción disparada	Estado	Saldo resultante	Folio	Test E2E correspondiente
R-01	No	--	--	A1	RECHAZADO	Sin cambio	None	test_pago_rechazado_concepto_invalido
R-02	Sí	No	--	A2	RECHAZADO	Sin cambio	None	test_pago_rechazado_monto_negativo
R-03	Sí	Sí	No	A3	RECHAZADO	Sin cambio	None	test_pago_rechazado_por_comision test_pago_rechazado_un_centavo_menos
R-04	Sí	Sí	Sí	A4	APROBADO	saldo-monto-\$15	Generado	test_pago_exitoso_renta test_pago_aprobado_saldo_exacto
R-05	No	Sí	Sí	A1	RECHAZADO	Sin cambio	None	test_saldo_inmutable_en_rechazos[Netflix-...]
R-06	No	No	Sí	A1	RECHAZADO	Sin cambio	None	test_saldo_inmutable_en_rechazos[Netflix-...]
R-07	No	No	No	A1	RECHAZADO	Sin cambio	None	test_saldo_inmutable_en_rechazos
R-08	Sí	No	Sí	A2	RECHAZADO	Sin cambio	None	test_saldo_inmutable_en_rechazos[Luz-...]

Nota: "--" indica que la condición no se evalúa por cortocircuito lógico (la condición anterior ya determinó el rechazo).

3.3 Análisis de valores límite (Condición C3)

El borde crítico del sistema ocurre cuando el saldo es exactamente igual al monto + \$15.00 de comisión. La siguiente tabla documenta los tres puntos de frontera y el comportamiento esperado:

Caso límite	Saldo	Monto	Total requerido	Resultado	Test
Por debajo del límite	\$1,014.99	\$1,000.00	\$1,015.00	RECHAZADO	test_pago_rechazado_un_centavo_menos

Caso límite	Saldo	Monto	Total requerido	Resultado	Test
Exactamente en el límite	\$1,015.00	\$1,000.00	\$1,015.00	APROBADO	test_pago_aprobado_saldo_exacto
Por encima del límite	\$5,000.00	\$3,500.00	\$3,515.00	APROBADO	test_pago_exitoso_renta

4. Código fuente -- payflow_pago.py

A continuación se presenta el código fuente completo del módulo de pagos, organizado siguiendo la arquitectura de tres capas definida en el planteamiento. Cada función incluye su docstring y pertenece exclusivamente a una capa.

4.1 Constantes y configuración

```
from datetime import datetime

COMISION_FIJA = 15.00
CONCEPTOS_VALIDOS = {"Renta", "Internet", "Luz"}
```

4.2 Capa Superior -- validar_pago()

```
def validar_pago(saldo: float, monto: float, concepto: str) -> dict:
    """
    Verifica si el pago puede procesarse según las reglas de negocio.
    Retorna: {"autorizado": bool, "motivo": str}
    """
    if concepto not in CONCEPTOS_VALIDOS:
        return {"autorizado": False,
                "motivo": f"Concepto '{concepto}' no válido."}
    if monto <= 0:
        return {"autorizado": False,
                "motivo": "El monto debe ser mayor a cero."}
    total_requerido = monto + COMISION_FIJA
    if saldo < total_requerido:
        return {"autorizado": False,
                "motivo": f"Fondos Insuficientes. Requerido: ${total_requerido:.2f}"}
    return {"autorizado": True, "motivo": "Saldo suficiente."}
```

4.3 Capa Inferior -- calcular_nuevo_saldo()

```
def calcular_nuevo_saldo(saldo: float,
                        monto: float,
                        comision: float) -> float:
    """
    Aritmética pura: nuevo_saldo = saldo - monto - comision
    No conoce reglas de negocio ni conceptos de servicio.
    """
    return round(saldo - monto - comision, 2)
```

4.4 Capa Media -- generar_folio()

```
def generar_folio(concepto: str, timestamp: str = None) -> str:
    """
    Genera el folio de comprobante: PAGO-
    Si no se proporciona timestamp usa datetime.now() (YYYYMMDDHHmmss).
    """
    if timestamp is None:
        timestamp = datetime.now().strftime("%Y%m%d%H%M%S")
    return f"PAGO-{concepto.upper()}{timestamp}"
```

4.5 Orquestador -- procesar_pago()

```
def procesar_pago(id_usuario: str, concepto: str,
                  monto: float, saldo: float) -> dict:
    """
    Flujo completo de pago en una sola llamada:
    1. Validar disponibilidad (Capa Superior)
    2. Ejecutar el descuento (Capa Inferior)
    3. Emitir comprobante (Capa Media)
    Retorna: {estado, nuevo_saldo, folio, motivo}
    """
    validacion = validar_pago(saldo, monto, concepto) # Paso 1
    if not validacion["autorizado"]:
        return {"estado": "RECHAZADO",
                "nuevo_saldo": saldo, # saldo inmutable
                "folio": None,
                "motivo": validacion["motivo"]}

    nuevo_saldo = calcular_nuevo_saldo(saldo, monto, COMISION_FIJA) # Paso 2
    folio = generar_folio(concepto) # Paso 3

    return {"estado": "APROBADO",
            "nuevo_saldo": nuevo_saldo,
            "folio": folio,
            "motivo": f"Pago de {concepto} exitoso."}
```

5. Suite de pruebas E2E -- test_pago_e2e.py

La suite está organizada en cuatro clases pytest, una por cada función/capa más los escenarios de integración end-to-end. En total contiene **25 tests** que cubren el 100% del código de payflow_pago.py.

Clase / Test	Escenario	Tipo	Porción (Slice)	Resultado esperado
TestValidarPago	Saldo suficiente → autorizado	Unitario	SL-01 análogo	autorizado = True
	Saldo insuficiente → rechazado	Valor límite	SL-02 análogo	'Fondos Insuficientes' en motivo
	Concepto inválido → rechazado	Unitario	SL-03	'no válido' en motivo
	Monto = 0 → rechazado	Error Guessing	--	autorizado = False
	Saldo exactamente igual a monto+\$15	Valor límite	VL-límite	autorizado = True
	Saldo un centavo menos	Valor límite	VL-abajo	autorizado = False
	Todos los conceptos válidos (parametrize)	Partición equiv.	--	autorizado = True × 3
TestCalcularNuevoSaldo	5000 – 3500 – 15 = 1485	Unitario	SL-01	1485.00
	Resultado redondeado a 2 decimales	Unitario	--	round(resultado, 2)
	Comisión independiente del concepto	Slicing / Bitácora	--	1485.00 siempre
TestGenerarFolio	Folio empieza con PAGO-	Unitario	--	startswith('PAGO-RENTA')
	Concepto en mayúsculas	Unitario	--	'INTERNET' en folio
	Timestamp fijo (stub)	Stub / Unitario	--	'PAGO-LUZ20260517120000'
	Sin timestamp → usa hora actual	Unitario	--	len(folio) > len('PAGO-RENTA')
TestProcesarPagoE2E	Pago exitoso Renta \$3,500	E2E / Integración	Escenario 1	APROBADO, saldo=1485, folio≠None
	Saldo \$1,010 no alcanza Internet \$1,000	E2E / Integración	Escenario 2	RECHAZADO, saldo=1010, folio=None
	Concepto 'Spotify' inválido	E2E / Integración	Escenario 3	RECHAZADO, saldo intacto, folio=None
	Saldo exactamente suficiente	Valor límite E2E	VL-límite	APROBADO, nuevo_saldo=0.00
	Saldo un centavo menos	Valor límite E2E	VL-abajo	RECHAZADO
	Monto negativo	Error Guessing E2E	--	RECHAZADO, folio=None
	Inmutabilidad: 3 rechazos distintos (parametrize)	Regresión / E2E	--	nuevo_saldo == saldo_original × 3

6. Análisis de Slicing -- Aislamiento de responsabilidades

El siguiente diagrama identifica qué líneas de código pertenecen exclusivamente a cada capa, aplicando el principio de separación de responsabilidades (SRP).

Capa	Funciones	Responsabilidad exclusiva	Lo que NO debe hacer	Líneas aprox.
Superior (Validación)	validar_pago()	Evaluar reglas de negocio: concepto válido, monto > 0, saldo ≥ monto + comisión. Devuelve solo un dict {autorizado, motivo}.	Restar dinero, generar texto de folio, conocer el valor de COMISION_FIJA más allá de recibirlo como parámetro implícito.	L. 14–28
Inferior (Balance)	calcular_nuevo_saldo()	Aritmética pura: saldo – monto – comision. Recibe tres números, devuelve un número redondeado a 2 decimales.	Conocer qué es 'Renta' o 'Internet'. Saber si el pago fue autorizado. Acceder a COMISION_FIJA directamente.	L. 31–37
Media (Comprobante)	generar_folio()	Formatear la cadena PAGO-. Opcionalmente recibe un timestamp externo (Stub en tests).	Decidir si el pago procede. Calcular saldos. Acceder al saldo o al monto directamente.	L. 40–47
Orquestador	procesar_pago()	Coordinar la secuencia: llamar a validar_pago → calcular_nuevo_saldo → generar_folio. Ensamblar el resultado final para el front-end.	Implementar lógica de negocio propia. Contiene únicamente llamadas a las capas y empaquetado del dict de salida.	L. 50–68

6.1 Respuesta a la pregunta de la bitácora de Slicing

Cómo se garantiza que la comisión de \$15.00 se aplica correctamente en la Capa Inferior sin que la Capa Superior tuviera que conocer el proceso matemático de resta?

La separación se logra mediante el **paso de parámetros como contrato entre capas**. La Capa Superior (validar_pago) solo evalúa la pregunta binaria *Alcanza el saldo?* usando la expresión `saldo < monto + COMISION_FIJA`. Si la respuesta es sí, devuelve `autorizado = True` y su trabajo termina ahí.

El orquestador (procesar_pago) entonces invoca a la Capa Inferior pasando explícitamente los tres argumentos numéricos: `calcular_nuevo_saldo(saldo, monto, COMISION_FIJA)`. La Capa Inferior recibe la comisión como un número ordinario y aplica la resta; no sabe que ese número representa una regla de negocio, un concepto de servicio, ni ningún contexto adicional.

Este diseño garantiza que si la comisión cambia (por ejemplo, a \$20.00), solo se modifica la constante `COMISION_FIJA` en un único lugar, y ambas capas la usan correctamente sin necesidad de refactorizar su lógica interna. La prueba `test_comision_es_independiente_del_concepto` valida explícitamente este aislamiento.

7. Ejecución de pruebas y cobertura

Una vez instaladas las dependencias en el entorno virtual, la suite se ejecuta con los siguientes comandos desde la carpeta raíz del proyecto:

7.1 Comandos de ejecución

Ejecución básica con detalle verboso:

```
$ pytest -v tests/test_pago_e2e.py
```

Ejecución con reporte de cobertura en terminal:

```
$ pytest --cov=src/payflow_pago tests/test_pago_e2e.py
```

Ejecución con reporte de cobertura y líneas faltantes:

```
$ pytest --cov=src/payflow_pago --cov-report=term-missing tests/test_pago_e2e.py
```

Generación de reporte HTML (análisis visual):

```
$ pytest --cov=src/payflow_pago --cov-report=html tests/test_pago_e2e.py
```

7.2 Criterios de aceptación de cobertura

Según el plan de pruebas PRA08, la suite debe cumplir los siguientes umbrales mínimos antes de emitir una recomendación **APROBADO**:

Criterio	Umbral mínimo	Resultado obtenido	Estado
Tests PASSED	25 / 25	25 / 25	✓ CUMPLIDO
Tests FAILED	0	0	✓ CUMPLIDO
Cobertura payflow_pago.py	≥ 90 %	100 %	✓ CUMPLIDO
Defectos S1-Crítico abiertos	0	0	✓ CUMPLIDO
Inmutabilidad del saldo ante rechazo	100 % escenarios	100 % (parametrize × 3)	✓ CUMPLIDO

8. Gestión de defectos

Durante el ciclo TDD de la PRA08 no se registraron defectos S1-Crítico ni S2-Mayor. La siguiente tabla documenta el inventario completo de incidencias encontradas y resueltas durante las fases Roja y Verde:

ID	Requerimiento	Capa	Síntoma observado	Severidad	Estado
D-01	C3 – saldo exacto	Superior	Operador '<' en lugar de '<=' causaba rechazo en saldo exactamente igual a monto+comisión	S2	CERRADO
D-02	Folio – mayúsculas	Media	concepto.upper() omitido; folio generado con minúsculas en el nombre del concepto	S3	CERRADO
D-03	Redondeo saldo	Inferior	Resultado con 4 decimales por omitir round(..., 2) en calcular_nuevo_saldo	S3	CERRADO

9. Bitácora de ciclos TDD

La siguiente tabla registra el ciclo Red-Green-Refactor para cada requerimiento del módulo PRA08, siguiendo el formato estándar del plan de pruebas.

Req.	Test escrito	Falló en Rojo?	Cambio para llegar a Verde	Hubo refactorización?	Responsable
C1 Concepto	test_concepto_invalido test_todos_los_conceptos_validos	Sí	Añadí verificación 'if concepto not in CONCEPTOS_VALIDOS' con retorno anticipado	Moví el catálogo a constante CONCEPTOS_VALIDOS = {set}	Guillermo
C2 Monto	test_monto_cero	Sí	Añadí 'if monto <= 0' antes de evaluar saldo	No	Marcos
C3 Saldo	test_saldo_insuficiente test_saldo_exacto test_un_centavo_menos	Sí	Implementé comparación 'if saldo < monto + COMISION_FIJA'; corrección de '<' a '>=' para el límite exacto	Sí -- renombré variable total a total_requerido para mayor claridad	Marcos
Balance (Inferior)	test_resto_correcta test_redondeo	Sí	Implementé 'return round(saldo - monto - comision, 2)'; añadí round() tras detectar D-03	No	Jorge
Folio (Media)	test_folio_empieza_PAGO test_timestamp_fijo	Sí	Implementé f-string 'PAGO-{concepto.upper()}{timestamp}'; añadí upper() tras detectar D-02	Sí -- parámetro timestamp=None para permitir Stub en tests	Guillermo
Orquestador E2E	test_pago_exitoso_renta test_pago_rechazado_por_comision test_concepto_invalido	Sí	Implementé procesar_pago() coordinando las 3 capas; retorno inmutable del saldo ante rechazo	Sí -- extraí verificación de saldo a validar_pago() para evitar lógica duplicada	Todos

10. Conclusiones

10.1 Sobre la arquitectura de tres capas

La separación en Capa Superior (validación), Capa Inferior (cálculo) y Capa Media (comprobante) demostró su valor durante el ciclo TDD: cada capa pudo probarse de forma aislada con tests unitarios antes de integrarse en el orquestador. Esto redujo el tiempo de diagnóstico de fallos, ya que un test rojo en `TestCalcularNuevoSaldo` apuntaba directamente a la aritmética sin necesidad de depurar la validación ni el formateo.

10.2 Sobre el valor de las pruebas E2E

Los 25 tests E2E cubrieron el 100% del código de producción e incluyeron escenarios que las pruebas unitarias aisladas no detectarían, como la inmutabilidad del saldo ante cualquier tipo de rechazo (concepto inválido, monto negativo o fondos insuficientes simultáneamente).

10.3 Sobre el ciclo TDD

El ciclo Red-Green-Refactor permitió detectar tres defectos (D-01, D-02, D-03) que habrían llegado a producción en un enfoque de pruebas posterior al desarrollo. En particular, el defecto D-01 (operador `<` vs `≥` en el límite exacto del saldo) es del tipo que los análisis de valores límite están diseñados para encontrar, y el test `test_saldo_exactamente_igual_a_monto_mas_comision` lo capturó en la primera ejecución en rojo.